#13 캠 가동 — 코드 상세 설명서

출처: Notion `[#13] 코드리뷰 · 캠 가동` (PR #112). 아래 코드는 Notion에 적힌 코드다.

1. 전체 흐름

```
관리자 화면: 캠 가동 버튼
   → CameraController.start
   → CameraStartService.start
        ① [커밋] 캠이 있고 STOPPED/ERROR인가? → run_requested = 1
        ② AnalyzerClient.startCamera   POST /analyzer/v1/cameras/{id}/start (5초)
        ③ [커밋] 성공: 상태 갱신 + first_started_at
                  실패: ERROR + 오류 내용
   → 200 {status}  /  409 / 404 / 422 / 502
```

2. 명세가 요구한 것 vs 구현

명세 (FNC-A-020)	구현
1. status가 STOPPED 또는 ERROR인지 확인	`CameraStartService.requestRun`, 아니면 `InvalidCameraStatusException` (409 `INVALID_STATUS`)
2. run_requested=1	`Camera.requestRun()`, 1단계 트랜잭션에서 커밋
3. 분석 서버 `POST /analyzer/v1/cameras/{id}/start` 호출(5초)	`AnalyzerClient.startCamera`
4. 응답의 상태로 status 갱신, first_started_at이 비어 있으면 현재 시각	`recordStarted` + `Camera.markFirstStartedIfAbsent`
5. 작업 기록(CAMERA_RUN, op: START)	**미구현** (#6 대기)
연결 실패 → ERROR('분석 서버 연결 실패'), 502, run_requested=1 유지	`AnalyzerUnavailableException` + `recordFailure`
400 SOURCE_NOT_FOUND → ERROR('영상 파일·스트림 없음'), 422, run_requested=1 유지	`SourceNotFoundException` + `recordFailure`
API: 200 `{"status": "PREPARING"}`	`CameraStartResponse`
후속: 캠 상태 SSE	이슈가 범위 밖(2단계)으로 정함

3. 코드

3-1. Camera (common) 두 메서드

```
/** 분석 서버가 이 캠을 가동해야 한다고 기록한다. 분석 서버가 다시 켜지면 이 값을 보고 자동 가동한다. */
public void requestRun() {
    this.runRequested = true;
}

/** 처음 가동에 성공한 시각을 기록한다. 이미 있으면 바꾸지 않는다. */
public void markFirstStartedIfAbsent(LocalDateTime now) {
    if (this.firstStartedAt == null) {
        this.firstStartedAt = now;
```

```
    }
}
```

상태를 바꾸는 길을 `Camera`의 메서드로만 열어 두었다. (setter 없음) "처음 가동한 시각은 한 번만 기록"이라는 규칙이 이 메서드 안에 있어서 호출하는 쪽이 실수할 수 없다.

3-2. ErrorCode와 설정

```
INVALID_STATUS(HttpStatus.CONFLICT, "현재 캠 상태에서는 처리할 수 없습니다."),
SOURCE_NOT_FOUND(HttpStatus.UNPROCESSABLE_ENTITY, "영상 파일·스트림을 찾을 수 없습니다."),
ANALYZER_UNAVAILABLE(HttpStatus.BAD_GATEWAY, "분석 서버에 연결할 수 없습니다."),
```

```
analyzer:
  base-url: ${ANALYZER_BASE_URL}

internal-api:
  token: ${INTERNAL_API_TOKEN}
```

- 명세의 오류 코드 이름과 HTTP 상태(409 / 422 / 502)를 그대로 따랐다. 이미 있던 `STATUS_CONFLICT`, `EXTERNAL_UNAVAILABLE`은 이름이 달라 쓰지 않았다.
- `ANALYZER_BASE_URL`, `INTERNAL_API_TOKEN`은 conventions.md '설정값'이 정한 이름이다. 값은 환경변수로만 넣고 `.env.example`에는 이름만 적었다.
- 팀원 #15 브랜치와 **같은 설정 이름·같은 줄**로 맞췄다. 합칠 때 충돌을 줄이려는 것이다.

3-3. 전화기: AnalyzerClient (camera/client)

```
@Component
@Log4j2
public class AnalyzerClient {

    private static final String INTERNAL_TOKEN_HEADER = "X-Internal-Token";
    private static final Duration START_TIMEOUT = Duration.ofSeconds(5);
    private static final Duration STOP_TIMEOUT = Duration.ofSeconds(15);
    private static final String SOURCE_NOT_FOUND_CODE = "SOURCE_NOT_FOUND";

    private final JsonMapper jsonMapper = JsonMapper.builder().build();
    private final RestClient startClient;
    private final RestClient stopClient;

    public AnalyzerClient(
            @Value("${analyzer.base-url}") String baseUrl, @Value("${internal-api.token}") String internalToke
n) {
        this.startClient = buildClient(baseUrl, internalToken, START_TIMEOUT);
        this.stopClient = buildClient(baseUrl, internalToken, STOP_TIMEOUT);
    }

    public CameraStatus startCamera(Long cameraId, CameraSourceType sourceType, String sourceUri) {
        Reply reply;
        try {
            reply = startClient
                    .post()
                    .uri("/analyzer/v1/cameras/{id}/start", cameraId)
                    .contentType(MediaType.APPLICATION_JSON)
                    .body(new StartRequest(sourceType, sourceUri))
                    .exchange((request, response) ->
                            new Reply(response.getStatusCode().value(), readJson(response.getBody())));
        } catch (RestClientException e) {
            log.warn("분석 서버 캠 시작 호출 실패 cameraId={}: {}", cameraId, e.getMessage());
            throw new AnalyzerUnavailableException(e);
        }

        if (reply.httpStatus() == 200) {
            return parseStartedStatus(cameraId, reply.body());
        }
```

```
        String errorCode = reply.body().path("code").asString("");
        if (reply.httpStatus() == 400 && SOURCE_NOT_FOUND_CODE.equals(errorCode)) {
            throw new SourceNotFoundException(cameraId);
        }
        log.warn("분석 서버가 캠 시작을 거부했습니다 cameraId={} http={} code={}", cameraId, reply.httpStatus(), errorCo
de);
        throw new AnalyzerUnavailableException();
    }

    // 캠 정지(FNC-A-021, #14)에서 정식으로 만든다. 팀원 #15가 쓰는 stop 호출만 둔다
    public void stopCamera(Long cameraId) { ... }

    private CameraStatus parseStartedStatus(Long cameraId, JsonNode body) {
        String status = body.path("status").asString("");
        if (CameraStatus.PREPARING.name().equals(status)) return CameraStatus.PREPARING;
        if (CameraStatus.RUNNING.name().equals(status)) return CameraStatus.RUNNING;
        log.warn("분석 서버가 알 수 없는 상태로 답했습니다 cameraId={} status={}", cameraId, status);
        throw new AnalyzerUnavailableException();
    }
    ...
}
```

부분	쉬운 뜻
`START_TIMEOUT` 5초 / `STOP_TIMEOUT` 15초	명세가 정한 시간. 시작은 5초 안에 답이 없으면 포기, 정지는 분석 서버가 종료를 최대 10초 기다리므로 15초
`X-Internal-Token` 헤더	관리자 앱 ↔ 분석 서버 요청에 붙이는 비밀 확인표. 값은 환경변수 `INTERNAL_API_TOKEN`
`.exchange(...)` 로 상태 코드와 본문을 직접 읽음	200 / 400 / 그 밖을 구분해야 하므로, 오류 응답에서 예외가 자동으로 튀어나오지 않게 하고 직접 읽는다
`RestClientException` → `AnalyzerUnavailableException`	연결 거부·응답 중 끊김·5초 초과는 모두 이 예외로 모인다 → 502
400 + `code=SOURCE_NOT_FOUND` → `SourceNotFoundException`	분석 서버가 "그 영상 파일이 없다"고 답한 경우만 따로 구분 (422)
그 밖의 오류 응답·알 수 없는 상태 값 → `AnalyzerUnavailableException`	명세가 정하지 않은 응답은 "분석 서버를 쓸 수 없음"으로 본다 (내가 정한 것, R4)
`log.warn(...)`	분석 서버 연결 실패는 WARN 로그
`readJson`이 본문이 비었거나 JSON이 아니면 빈 노드 반환	본문이 이상해도 예외가 엉뚱한 곳에서 터지지 않게 한다

3-4. 총무: CameraStartService (핵심)

```
@Service
@RequiredArgsConstructor
public class CameraStartService {

    private static final String UNAVAILABLE_MESSAGE = "분석 서버 연결 실패";
    private static final String SOURCE_NOT_FOUND_MESSAGE = "영상 파일·스트림 없음";
    private static final int PREPARING_START_PROGRESS = 0;

    private final CameraRepository cameraRepository;
    private final AnalyzerClient analyzerClient;
    private final TransactionTemplate transactionTemplate;

    public CameraStartResponse start(Long cameraId) {
        // 처리 순서 1·2: STOPPED·ERROR인지 확인하고 run_requested=1 기록
        StartTarget target = transactionTemplate.execute(status -> requestRun(cameraId));

        // 처리 순서 3: 분석 서버 호출. 실패해도 run_requested=1은 유지한다
        CameraStatus started;
        try {
            started = analyzerClient.startCamera(cameraId, target.sourceType(), target.sourceUri());
```

```
        } catch (AnalyzerUnavailableException e) {
            recordFailure(cameraId, UNAVAILABLE_MESSAGE);
            throw e;
        } catch (SourceNotFoundException e) {
            recordFailure(cameraId, SOURCE_NOT_FOUND_MESSAGE);
            throw e;
        }

        // 처리 순서 4: 응답의 상태로 갱신, first_started_at이 비어 있으면 현재 시각
        recordStarted(cameraId, started);
        // 처리 순서 5(작업 기록 CAMERA_RUN, detail: {op: START})는 #6이 들어오면 여기서 호출한다
        return new CameraStartResponse(started);
    }

    private StartTarget requestRun(Long cameraId) {
        Camera camera = cameraRepository.findByIdAndDeletedAtIsNull(cameraId)
                .orElseThrow(() -> new CameraNotFoundException(cameraId));
        if (!isStartable(camera.getStatus())) {
            throw new InvalidCameraStatusException(cameraId, camera.getStatus());
        }
        camera.requestRun();
        return new StartTarget(camera.getSourceType(), camera.getSourceUri());
    }

    private void recordFailure(Long cameraId, String errorMessage) {
        transactionTemplate.executeWithoutResult(status ->
                cameraRepository.findByIdAndDeletedAtIsNull(cameraId)
                        .ifPresent(camera -> camera.updateStatus(CameraStatus.ERROR, null, errorMessage)));
    }

    private void recordStarted(Long cameraId, CameraStatus started) {
        transactionTemplate.executeWithoutResult(status ->
                cameraRepository.findByIdAndDeletedAtIsNull(cameraId).ifPresent(camera -> {
                    // 분석 서버의 상태 보고(FNC-A-025)가 응답보다 먼저 와서 이미 상태가 바뀌었다면 덮어쓰지 않는다
                    if (isStartable(camera.getStatus())) {
                        Integer progress = started == CameraStatus.PREPARING ? PREPARING_START_PROGRESS : null;
                        camera.updateStatus(started, progress, null);
                    }
                    camera.markFirstStartedIfAbsent(LocalDateTime.now());
                }));
    }

    private boolean isStartable(CameraStatus status) {
        return status == CameraStatus.STOPPED || status == CameraStatus.ERROR;
    }
}
```

부분	쉬운 뜻
메서드에 `@Transactional`이 없다	**일부러 뺐다.** 전체를 한 묶음으로 만들면 ① 분석 서버를 최대 5초 기다리는 동안 DB 연결을 붙들고 ② 실패해서 예외를 던질 때 "ERROR로 바꾼 것"까지 함께 취소된다. 명세는 실패해도 ERROR와 run_requested=1을 남기라고 했으므로 두 번째가 치명적이다
`transactionTemplate.execute` / `executeWithoutResult`	"여기서부터 여기까지만 한 묶음으로 커밋"을 코드로 직접 정한다. 세 번 따로 커밋한다
`requestRun`에서 예외가 나면	그 묶음만 취소된다 (404·409). 아직 아무것도 바꾸지 않았고 분석 서버도 부르지 않는다
실패 시 `recordFailure` 후 `throw e`	ERROR를 먼저 확정(커밋)한 뒤 예외를 다시 던져 502·422 응답이 나가게 한다
`if (isStartable(camera.getStatus()))`	분석 서버의 상태 보고(#15)가 우리 3단계보다 먼저 도착해 이미 RUNNING으로 바뀌었는데 우리가 PREPARING으로 덮어쓰는 일을 막는다. `first_started_at`은 그래도 채운다
`PREPARING`이면 progress를 0으로, RUNNING이면 null 전달	이전 ERROR 때 남은 진행률(예: 40%)이 새 준비 단계에 보이지 않게 한다
오류 메시지 두 개를 상수로	명세가 문구를 정했다: '분석 서버 연결 실패', '영상 파일·스트림 없음'

부분	쉬운 뜻
`updateStatus(started, progress, null)`	세 번째 인자가 null이라 이전 오류 내용이 지워진다 (ERROR에서 다시 가동하는 경우)

3-5. 경비원과 응답 상자

```
// CameraController에 추가
@PostMapping("/{id}/start")
public CameraStartResponse start(@PathVariable Long id) {
    return cameraStartService.start(id);
}

// CameraStartResponse.java
public record CameraStartResponse(CameraStatus status) {}

// 예외 4종 (모두 BusinessException 상속)
InvalidCameraStatusException → ErrorCode.INVALID_STATUS   (409)
SourceNotFoundException      → ErrorCode.SOURCE_NOT_FOUND (422)
AnalyzerUnavailableException → ErrorCode.ANALYZER_UNAVAILABLE (502)
CameraNotFoundException      → ErrorCode.NOT_FOUND (404)  // 팀원 #15 브랜치의 임시 코드와 같은 내용
```

컨트롤러는 받아서 서비스에 넘기기만 한다. 성공하면 기본 200이다. 오류는 던지기만 하면 공통 오류 처리(FNC-C-027)가 `{code, message}` 응답으로 바꿔 준다.

4. 테스트 12개 (전부 통과)

분석 서버는 같은 JVM에 띄운 **가짜 HTTP 서버**(`com.sun.net.httpserver.HttpServer`)로 대신한다. 응답 코드·본문·지연·연결 끊기를 테스트마다 정하고, 마지막으로 받은 요청(경로·토큰·본문)을 기록한다.

테스트	확인하는 것
`start_stoppedDummyCamera_becomesPreparing`	200 `{status: PREPARING}`, DB: PREPARING, progress=0, 오류 내용 없음, run_requested=1, first_started_at 채워짐
`start_sendsAnalyzerRequest`	분석 서버가 받은 요청: POST, 경로, X-Internal-Token, 본문 sourceType·sourceUri
`start_analyzerReportsRunning_becomesRunning`	분석 서버가 RUNNING으로 답하면 캠이 RUNNING
`start_errorCamera_clearsErrorMessage`	ERROR(진행률 40, 오류 내용 있음)에서 재가동 → PREPARING, 오류 내용 null, 진행률 0
`start_keepsFirstStartedAt`	이미 first_started_at이 있으면 바꾸지 않음
`start_notStartable_conflict`	PREPARING·RUNNING이면 409, 분석 서버 호출 0회, 상태·run_requested 그대로
`start_analyzerConnectionFails_badGatewayAndError`	연결이 끊기면 502, ERROR·'분석 서버 연결 실패', run_requested=1 유지, first_started_at 비어 있음
`start_analyzerTimesOut_badGateway`	분석 서버가 7초 뒤에 답하게 해 두고 약 5초(4.5~6.8초) 안에 502로 끝나는지, 캠이 ERROR가 되는지 확인 (가장 느린 테스트, 약 6.7초)
`start_sourceNotFound_unprocessableAndError`	400 SOURCE_NOT_FOUND → 422, ERROR·'영상 파일·스트림 없음', run_requested=1 유지
`start_otherAnalyzerErrors_badGateway`	500·401·501·400(다른 코드)·HTML 본문 → 모두 502와 ERROR
`start_unexpectedSuccessBody_badGateway`	200인데 상태가 STOPPED / 빈 본문 / JSON 아님 → 502와 ERROR
`start_unknownOrDeletedCamera_notFound`	없는 캠·삭제된 캠 → 404, 분석 서버 호출 0회

5. 왜 이렇게 만든 게 맞는가

결정	이유	근거
트랜잭션을 세 단계로 쪼갬	실패 상태(ERROR)가 예외와 함께 롤백되면 명세의 "ERROR, run_requested=1 유지"를 못 지킨다. 분석 서버를 기다리는 동안 DB도 안 붙든다	FNC-A-020 예외 상황
run_requested=1을 호출 전에 먼저 커밋	호출이 실패하거나 앱이 중간에 죽어도 "켜야 한다"는 기록이 남아 분석 서버가 다시 켜질 때 자동 가동된다	FNC-A-020 처리 순서 2
분석 서버 호출은 `camera/client/AnalyzerClient` 한 곳	다른 서버 호출은 client에만 둔다 (service는 판단만)	package-structure.md '계층 규칙'
팀원 #15의 임시 AnalyzerClient와 클래스·설정 이름·시그니처를 맞춤	같은 파일 경로이므로 이름이 다르면 합칠 때 충돌·중복 설정이 생긴다	#15 브랜치의 TODO(#13)
오류는 `BusinessException` + `ErrorCode`	공통 오류 처리가 응답을 만든다. 전용 핸들러가 필요 없다	FNC-C-027
가짜 분석 서버로 테스트	실제 분석 서버(flask #1)가 없어도 요청 내용·응답별 처리·타임아웃을 확인할 수 있다	CLAUDE.md '검수 기준은 테스트로'

6. 선배가 물을 만한 질문과 답

질문	답
"왜 `@Transactional`을 안 붙였어? #7·#10은 붙였잖아"	여기는 외부 서버(분석 서버)를 최대 5초 기다린다. 한 묶음이면 그동안 DB를 붙들고, 실패 때 ERROR로 바꾼 것까지 롤백된다. 명세는 실패해도 ERROR와 run_requested=1을 남기라고 해서 `TransactionTemplate`으로 세 단계로 나눴다.
"`run_requested=1`을 왜 먼저 커밋해?"	호출이 실패하거나 앱이 죽어도 "켜야 한다"는 기록이 남아야, 분석 서버가 다시 켜질 때 그 기록을 보고 자동 가동하기 때문이다.
"분석 서버가 응답하기 전에 상태 보고가 먼저 오면?"	3단계에서 아직 STOPPED/ERROR일 때만 상태를 바꾼다. 이미 RUNNING이면 덮어쓰지 않는다.
"ERROR였던 캠을 다시 켜면 이전 진행률이 남지 않아?"	PREPARING이면 progress를 0으로, 오류 내용은 null로 덮어서 지운다.
"명세에 없는 401, 501은 어떻게 처리해?"	내가 정해서 모두 502 + ERROR('분석 서버 연결 실패')로 처리했다. 특히 LIVE를 가동하면 501이 와서 "연결 실패"로 보일 수 있어 오해 소지가 있다. leads와 정하자고 제안했다.
"분석 서버를 실제로 붙여 봤어?"	아니다. 가짜 HTTP 서버로 요청 내용·타임아웃·오류별 처리를 확인했다. PREPARING → RUNNING 전환은 상태 보고(#15)와 flask #1이 들어와야 확인할 수 있다.
"같은 캠을 동시에 두 번 가동하면?"	둘 다 상태 확인을 통과해 분석 서버를 두 번 부른다. 분석 서버가 "이미 돌고 있으면 현재 상태로 200"을 주는 멱등 API라 결과는 같지만 잠금은 없다.
"5초 타임아웃은 어디서 정했어?"	명세(api/analyzer.md)가 정했다. 정지는 분석 서버가 종료를 최대 10초 기다려서 15초다.

7. 명세 대조 (FNC-A-020)

명세 항목	구현	확인
1. STOPPED·ERROR 확인, 아니면 409	`requestRun`	`start_notStartable_conflict`, `start_errorCamera_clearsErrorMessage`

명세 항목	구현	확인
2. run_requested=1	`Camera.requestRun`	`start_stoppedDummyCamera_becomesPreparing`, 실패 테스트들
3. 분석 서버 start 호출(5초)	`AnalyzerClient.startCamera`	`start_sendsAnalyzerRequest`, `start_analyzerTimesOut_badGateway`
4. 응답 상태로 갱신, first_started_at	`recordStarted`	`..._becomesPreparing`, `..._becomesRunning`, `start_keepsFirstStartedAt`
5. 작업 기록	**미구현** (#6 대기)	-
연결 실패: ERROR·502·run_requested 유지	`recordFailure` + `AnalyzerUnavailableException`	`start_analyzerConnectionFails_badGatewayAndError`
400 SOURCE_NOT_FOUND: ERROR·422·run_requested 유지	`SourceNotFoundException`	`start_sourceNotFound_unprocessableAndError`
완료 기준: PREPARING → RUNNING 전환	PREPARING 또는 RUNNING 응답을 반영 (전환 자체는 상태 보고 필요)	R2
후속: 캠 상태 SSE	이슈 범위 밖 (2단계)	-